Anumati: Proof of Adherence as a Formal Consent Model for Autonomous Agent Protocols

Ravi Kiran Kadaboina

Independent Researcher

Anumati is Sanskrit for "consent" or "formal permission."

Abstract

As autonomous AI agents increasingly call other agents to complete tasks, a structural accountability gap has emerged: agents accept the terms of service of the services they call without any mechanism to prove they understood or subsequently honoured those terms. Authentication protocols establish *who* may call *what*. They say nothing about *under what conditions* a permitted call may be made, conditions that change as policies evolve. We formalise this as the distinction between *proof of acceptance* (a timestamped acknowledgement) and *proof of adherence* (a clause-level reasoning record produced at every action). We propose three primitives — PolicyDocument, ConsentRecord, and AdherenceEvent — that together constitute a versioned, append-only consent model for agent-to-agent communication. We instantiate the model as concrete extensions to two major agent protocols: the Agent2Agent (A2A) protocol and the Model Context Protocol (MCP). The model directly addresses the consent obligations of the EU AI Act ahead of the August 2, 2026 enforcement date for high-risk AI systems (Arts. 14, 50) and fills an identified gap in the FINOS AI Governance Framework.

---

1. Introduction

Terms of service are a legal fiction. In practice, users click "I agree" without reading, companies record the click, and everyone moves on. The acceptance is real; the comprehension is not. Legal systems have accommodated this fiction for decades because there was no better option. Humans cannot be expected to read twenty-page policies before accessing a service.

Several years ago, the author implemented a versioned terms-of-service system at a consumer technology company that tried to take this problem seriously. Policy documents were managed in a headless CMS with full version history; each published version was immutable and content-addressed. The auth state machine (built on a custom OAuth 2-based flow) checked on every session whether the user's most recently accepted document version matched the current published version. If not, the session was blocked and the user was presented with the new terms before proceeding. Every acceptance event was stored with a pointer

to the exact document version and a reference to the prior acceptance record, forming a singly-linked chain per user. The goal was legal defensibility: in any dispute, the system could show precisely which version of the terms a user had accepted, and when, across the full history of their account.

What the data showed was unsurprising in hindsight: median time on the acceptance screen was under four seconds. Users were not reading. The linked chain proved acceptance with precision; it proved nothing about comprehension or subsequent compliance. The gap between the audit trail we could produce and the assurance it actually gave is the motivation for the model in §3.

That system worked, within its limits. The limits were human ones. But the failure mode was not what we expected. Users were not harmed by their failure to read. They relied on something else entirely: intuition about trust. A user accepting terms on apple.com does not read the document. They do not need to. Apple's brand, its public stance on privacy, its App Store review policies, its Safari tracking protections. These are signals the user processes instantly and unconsciously. The terms are a legal formality; the trust decision was made before the page loaded. In this context, human intuition actually works *in favour* of the user.

AI agents have no equivalent intuition. An agent calling a callee service for the first time has no brand association, no accumulated trust signal, no instinct about whether this service is "the kind of service that honours its terms." Worse, even if an agent develops something resembling learned trust (through fine-tuning, retrieval-augmented context, or long-term memory), that knowledge is fragile. A new task, a new capability, a shifted context window can override whatever the agent previously learned about who to trust and why. Trust based on learned associations does not survive capability drift.

That absence is what the model in §3 addresses. The answer is not to make agents read verbose legal documents the way humans fail to. It is to replace the verbose document with precisely curated, machine-readable policy claims: structured constraints that agents can evaluate deterministically at every action, independent of learned trust or contextual memory. The claims are not suggestions. They are hard boundaries that produce an audit trail whether the agent "trusts" the callee or not.

The inadequacy of this model is no longer theoretical. The 2025 AI Agent Index surveyed 30 deployed agents and found that only 6 explicitly stated that their crawler bots respect robots.txt; 16 provided no clear statement about web exclusion compliance at all [1]. Robots.txt is a proxy: if agents ignore the most basic signal of "do not access this," there is no reason to expect they will honour more nuanced usage terms. The problem is empirical and worsening as agent deployment scales. Meanwhile, the EU AI Act enters its third enforcement phase on August 2, 2026, when obligations for high-risk AI systems under Art. 14 (human oversight) and Art. 50 (transparency) take effect, imposing requirements that extend to autonomous agent transactions with no published

technical guidance on what that means in practice [2, 3]. (Earlier phases — prohibited AI practices from February 2025, general-purpose AI from August 2025 — are already in force.)

This paper makes three contributions:

- A formal account of why authentication-layer controls are insufficient for usage policy governance in agent-to-agent communication (§2).
- A minimal three-primitive consent model — PolicyDocument, ConsentRecord, and AdherenceEvent — that produces proof of adherence rather than proof of acceptance, with a versioned linked-list audit chain derived from production experience in human auth systems (§3).
- Concrete instantiations of the model as extensions to the Agent2Agent (A2A) protocol [4] and the Model Context Protocol (MCP) [5], showing the gap is systemic across the agent stack (§4).

A reference implementation, including the TLA+ specification and protobuf schema, is in preparation at: https://github.com/ravikiran438/agent-consent-protocol

---

2. The Consent Gap in Agent Protocols

2.1 What Authentication Covers (and What It Does Not)

Modern agent protocols delegate security to standard HTTP authentication mechanisms. A2A supports OAuth 2.x, API keys, OpenID Connect, and mutual TLS [4]. MCP uses OAuth 2.1 with incremental scope negotiation following its November 2025 specification update [6]. Both are well-designed for their stated purpose: establishing *who* may invoke *which* capability.

Usage policy governs something different. Consider a callee agent that provides a data analysis service. Authentication tells the calling agent whether it is permitted to invoke the `analyse_dataset` skill. It says nothing about whether the calling agent:

- may store the task output beyond the current session;
- may aggregate results across multiple principals;
- is obligated to notify its human principal before invoking skills above a cost threshold; or
- may share results with a third agent the callee has not authorised.

These constraints are contextual, compositional, and they change as the callee's policies evolve. They cannot be encoded in OAuth scopes, which are discrete, static, and binary. Scopes answer "permitted or not." Usage policy answers "permitted, under these conditions, as of this version, subject to change."

2.2 The Version Problem

Human-facing terms of service change. Services update their policies, publish new versions, and expect users to re-accept. In practice, this is handled by a login gate: the user sees a banner, clicks agree, and the session proceeds. The mechanism is crude but functional.

For agents, no equivalent mechanism exists in current protocols. The A2A Agent Card contains `tos_uri` and `policy_uri` fields, display metadata not enforceable constraints [4]. When a callee updates its policy, calling agents have no protocol-level notification, no structured way to record re-acceptance, and no mechanism to block skill invocation until the new policy has been processed.

This creates a liability accumulation problem. Under the Uniform Electronic Transactions Act (UETA) §14, contracts formed by electronic agents bind the human principal, even without their awareness [7, 8]. As callee policies evolve and calling agents continue operating under stale terms, the principal's legal exposure compounds silently.

2.3 Proof of Acceptance vs. Proof of Adherence

We distinguish two consent properties:

Proof of acceptance is the traditional model: a timestamped record binding an identity to a document version at a point in time. It proves a party *agreed*. It proves nothing about whether they *understood* or subsequently *complied*.

Proof of adherence is what agents can uniquely provide: a per-action record citing the specific policy clause evaluated, the agent's reasoning, and the enforcement decision. It proves the agent *evaluated* the relevant clause *before acting*.

Table 1 summarises the distinction.

Dimension	Proof of Acceptance	Proof of Adherence
Granularity	Whole document	Per-clause, per-action
Timing	At acceptance event	At every action attempt
Content	Timestamp + identity	Clause citation + reasoning
Understanding	Assumed	Verified (parsed claims)
Post-acceptance compliance	Unverifiable	Auditable
Legal value	Proves agreement	Proves agreement *and* compliance

The gap between the two rows is what this paper addresses. Agents can produce proof of adherence because they can reason about policy text; the consent infrastructure simply does not ask them to.

3. A Formal Consent Model

We define three primitives. Together they constitute a versioned, append-only consent model for agent-to-agent communication. We refer to the resulting protocol as the **Agent Consent and Adherence Protocol (ACAP)**.

3.1 PolicyDocument

A `PolicyDocument` is the machine-readable equivalent of a terms-of-service document. It is versioned using semantic versioning, content-addressed via SHA-256 hash, and published by the callee agent at a well-known HTTPS URL.

Formally, a `PolicyDocument` P is a tuple:

```
P = (version, hash, effective_date, supersedes, claims,
publisher, tos_uri)
```

where `version` is a semver string, `hash` is the SHA-256 digest of the canonical JSON serialisation of P with the `hash` field set to the empty string (breaking the circularity: the digest input never includes itself), `effective_date` is an ISO 8601 timestamp, `supersedes` is the version of the document P replaces (if any), `claims` is a sequence of `PolicyClaim` objects (order-preserving; claim order is significant for display and diff computation), `publisher` is the callee agent's identifier (DID or HTTPS URL), and `tos_uri` is the URL of the human-readable document from which the claims are derived.

A `PolicyClaim` c is a tuple:

```
c = (id, clause_ref, action, asset, rule_type, constraint,
since_version,
     category, dimension)
```

where `rule_type` $\in$ {permission, prohibition, obligation}, `action` and `asset` use ODRL 2.2 vocabulary where applicable [9], and `constraint` is an optional ODRL constraint expression. The `since_version` field records which policy version introduced this claim, enabling calling agents to compute diffs between versions.

The key design constraint: every `PolicyClaim` must have a stable `id` across versions. A claim that changes meaning between versions MUST be assigned a new `id`; the old `id` is retired. This allows calling agents to detect precisely which claims changed when a version bumps.

3.2 ConsentRecord

A `ConsentRecord` documents the calling agent's parsed understanding of and decision about a specific `PolicyDocument` version. Records form a singly-linked list (the *consent chain*) for a given caller-callee pair.

Formally, a `ConsentRecord` R is a tuple:

```
R = (id, prev_id, caller, callee, policy_version, policy_hash,
     parsed_claims, decision, timestamp, valid_until, signature,
     policy_diff, caller_capability_hash, reconsent_trigger,
     category_preferences)
```

where `prev_id` references the immediately preceding record in the chain (null for the first record), `parsed_claims` contains one `ParsedClaim` entry for every `PolicyClaim` in the referenced `PolicyDocument`, `decision` ∈ {accepted, rejected, conditional}, and `valid_until` is either an ISO 8601 timestamp or a sentinel string value (`"on_version_bump"`, `"on_capability_change"`, or `"on_any_change"` — see §6.3).

A `ParsedClaim` entry records, for each policy claim: whether the calling agent understood the claim, whether it disputes the claim, and if disputed, a natural-language explanation.

Critical invariant: every `PolicyClaim` in the `PolicyDocument` MUST have a corresponding `ParsedClaim` in the `ConsentRecord`. Agents cannot silently ignore inconvenient clauses. This single requirement is what separates the model from a boolean acceptance flag.

Conditional consent and reduced permissions. When `decision` is `conditional`, the calling agent has accepted *some* claims but disputes others. The callee MUST enforce claim-level gating: a skill whose governing `PolicyClaim` is marked `disputed: true` in the `ParsedClaim` array is blocked, while skills governed by undisputed claims remain invocable. This allows the agent to continue operating at reduced capability rather than halting entirely. This is a practical necessity for long-running agent workflows where a blanket rejection would cascade into downstream failures.

Callee-side enforcement of conditional consent. The callee enforces claim-level gating by cross-referencing the `ParsedClaim` array in the caller's `ConsentRecord` against its own skill registry. Each skill in the callee's A2A `AgentCard` (or MCP tool manifest) is annotated with the `claim_id` values that govern it via the `policy_claims` array (see §4.1). When the callee receives a task request, it resolves the caller's active `ConsentRecord`, iterates the requested skill's `policy_claims`, and checks whether any referenced claim is marked `disputed: true` or `understood: false` in the corresponding `ParsedClaim`. If so, the callee MUST reject the invocation with a structured error indicating which claims block the skill. This enforcement is stateless from the callee's perspective — it requires only the caller's current `ConsentRecord` and the skill's `policy_claims` mapping, and can be implemented as middleware on the `POST /acap/adherence` endpoint (§4.1).

Policy diff on version bump. When a calling agent re-consents after a version bump, the `ConsentRecord` MUST include a `PolicyDiff`: a structured comparison of the previous and current `PolicyDocument` versions. The diff is computed mechanically: the agent compares `claim_id` sets between versions, using `since_version` to identify added claims, retired `claim_id` values to iden-

tify removals, and field-level comparison for modifications. Each `ClaimDiff` entry carries a natural-language `summary` generated by the agent (e.g., "§3.4 added: prohibition on aggregating session data for behavioural profiling"). The `PolicyDiff` also includes an `impact_summary`: one-paragraph assessment of how the changes affect the caller's current operations.

Escalation to the human principal. When the calling agent encounters a claim it does not understand (`understood: false`), disputes a claim, or the claim carries `escalate_on_deny: true`, the agent SHOULD present the `PolicyDiff` to its human principal rather than the raw policy document. The diff is the mechanism that makes human escalation *actionable*: instead of "we updated our terms, click agree," the principal sees "§3.4 was added, which prohibits session aggregation for profiling, which affects your current analytics workflow." The principal's response (accept, reject, or override) is recorded in the `ConsentRecord` and attributed via `principal_id`.

The linked-list structure is central to the legal value of the model. A chain of records shows the full history of what a calling agent agreed to on behalf of its principal, and what it agreed to *at the time of each action*, not just at signup. This pattern was validated in a prior production human-auth system where the same linked structure was used to preserve all accepted versions of terms per user for legal auditability. The contribution here is applying the pattern to the A2A context and extending it to the per-action record that follows.

3.3 AdherenceEvent

An `AdherenceEvent` records the calling agent's runtime evaluation of a policy claim for a specific action attempt. Events form a second singly-linked list (the *adherence trail*) anchored to a `ConsentRecord`.

Formally, an `AdherenceEvent` E is a tuple:

```
E = (id, prev_id, consent_record_id, action, claim_id,
clause_ref,
     decision, reasoning, timestamp, context, signature)
```

where `decision` $\in$ {permit, deny, escalate} and `reasoning` is a natural-language string explaining why the agent reached that decision.

The `reasoning` field is the mechanism that makes adherence auditable. An agent that records:

> *"Action 'aggregate_sessions' maps to odrl:aggregate on pii:session_data. Policy v2.1.0 §3.4 prohibits this where purpose = behavioural_profiling. Denying."*

has produced something qualitatively different from a system that simply refuses a request. The reasoning is citable, inspectable, and attributable. Auditors can trace any enforcement decision back to the exact clause that governed it.

3.4 Chain Properties

The two linked lists (consent chain and adherence trail) together satisfy five properties relevant to legal and regulatory accountability:

1. **Completeness.** Every action attempt produces an adherence event. A deny without an event is indistinguishable from a silent failure.

2. **Traceability.** Every adherence event references a consent record, which references a policy document version. The full chain from action to clause is traversable.

3. **Tamper evidence.** Both records and events carry optional JWS signatures [10] over their canonical JSON with the `signature` field set to the empty string prior to signing (the same bootstrapping convention used for `PolicyDocument.hash` in §3.1). The resulting signature is then inserted into the record. A verifier can confirm no record was modified after signing.

4. **Version fidelity.** `policy_hash` in the consent record binds the acceptance to the exact document content, independent of URI availability. The hash survives link rot.

5. **Ledger anchoring (optional).** JWS signatures prove *who signed* but not *when*, and they cannot prevent collusion. If both parties agree to rewrite the chain, signatures alone cannot detect it. For deployments requiring third-party tamper evidence, each record may carry a `ChainAnchor`: a cryptographic hash anchored to an external append-only ledger. The model is ledger-agnostic; the anchor may point to a public blockchain (Ethereum, Polygon), a permissioned ledger (Hyperledger Fabric), or a transparency log (RFC 9162).

   Three anchoring strategies are defined to balance assurance against cost:

Strategy	What is anchored	When	Cost profile
Per-record	Every consent record and adherence event individually	On creation	Highest assurance, highest cost
Merkle batch	Merkle root of a batch of records (batch closes on a fixed interval, e.g. hourly, or when batch size reaches a threshold, e.g. 100 events), anchored periodically	On schedule or threshold	Moderate assurance, low cost
Consent-only	Only consent records (not adherence events)	On consent	Consent events are infrequent but legally significant

   The anchor records the `record_hash`, the `ledger_id`, the `transaction_id`,

and a `verification_url` where any party can independently confirm the anchor. This means a regulator or auditor does not need to trust either agent. They can verify the anchor against the public ledger directly.

Anchoring is strictly additive: it strengthens the tamper-evidence property (3) without changing the protocol flow. Deployments that do not require it simply omit the `chain_anchor` field.

3.5 Tiered Escalation via Governance Agent

Not every policy change warrants human attention. A callee that fixes a typo in its terms, tightens a constraint, or adds a claim that aligns with the caller's existing mandate should not trigger a modal dialog interrupting a human operator. Treating every re-consent event as equally urgent produces alert fatigue, the same failure mode as click-through consent, relocated from the acceptance screen to the notification queue.

The model introduces a *governance agent*: a designated supervisory agent that evaluates the materiality of changes before deciding whether to escalate to the human principal. This pattern draws directly from the Governance-as-a-Service (GaaS) architecture [19], which factors compliance evaluation into a dedicated agent layer; here we specialise it to the narrower problem of consent-change materiality assessment. The escalation path is shown in Figure 1.

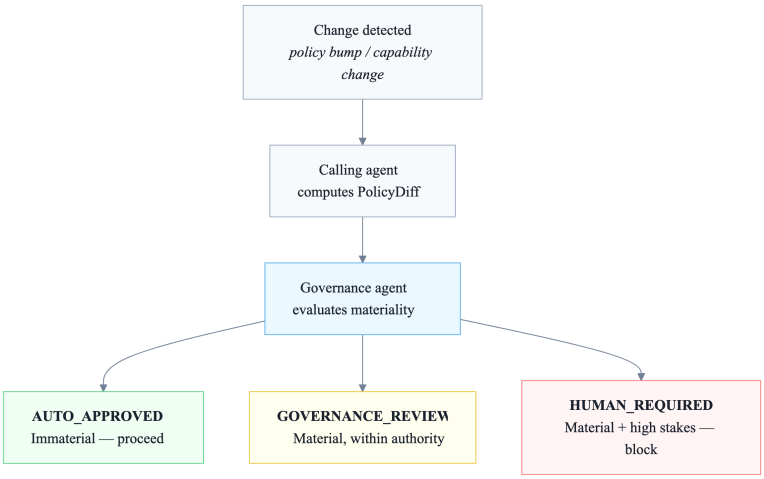

Figure 1: Tiered escalation via governance agent.

The governance agent's assessment is recorded as an `EscalationAssessment` on the `ConsentRecord`, containing:

- **Materiality signals:** individual factor-level assessments (e.g., "new prohibition added" → material; "clause_ref renumbered" → immaterial) with

natural-language reasoning for each.

- **Escalation tier:** the governance agent's overall decision.
- **Governance agent identity:** DID of the agent that performed the assessment, with optional JWS signature.
- **Principal notification flag:** whether the human was notified (some regulatory contexts require notification even for auto-approved changes).

Materiality criteria. The governance agent evaluates at minimum:

Factor	Material if...
New claim added	`rule_type` = prohibition or obligation
Claim removed	Any removed permission (capability loss)
Constraint changed	Constraint relaxed (e.g., data use expanded)
`rule_type` changed	Permission → obligation, or any → prohibition
`escalate_on_deny`	Any claim carries `escalate_on_deny: true`
Capability change	Model family changed (not just patch version)

Delegation boundary. The governance agent operates under a pre-configured delegation policy from the human principal, effectively a meta-policy that defines what the governance agent may auto-approve. This policy is outside the scope of ACAP (it governs the principal-to- governance-agent relationship, not the caller-callee relationship), but the `EscalationAssessment` records which tier was applied and why, ensuring the delegation decision is auditable.

This design preserves the core guarantee: every re-consent event produces a `ConsentRecord` with full `ParsedClaim` coverage. The governance agent does not skip claims. It determines who *reviews* the claims. The human principal retains override authority at all tiers.

3.6 Formal Verification of the Consent Lifecycle

We specify the consent lifecycle as a TLA+ state machine and verify its safety and liveness properties under model checking. The specification is included in the reference repository (`specification/ConsentLifecycle.tla`); publication of the repository is in preparation alongside this preprint.

The lifecycle has seven states for a given caller–callee pair, shown in Figure 2.

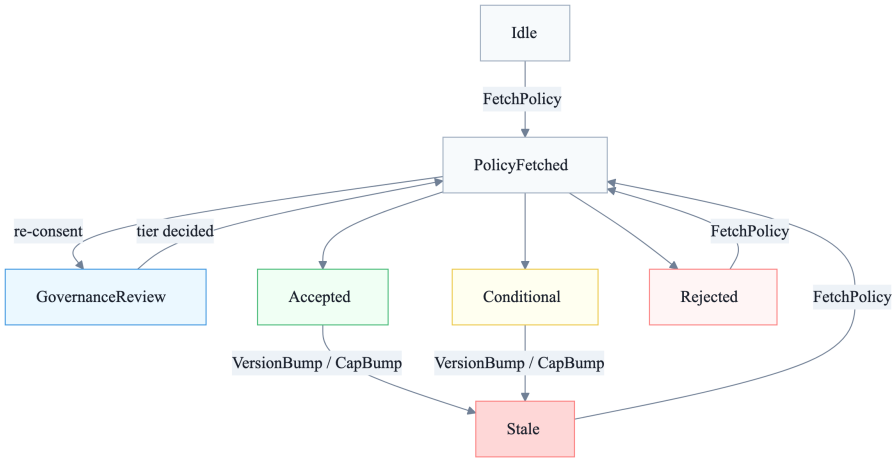

Figure 2: ACAP consent lifecycle state machine.

Table 2. Safety and liveness properties verified by TLC.

ID	Property	Kind	Statement
S1	NoSkillWithoutConsent	Safety	A skill call never occurs unless at least one ConsentRecord in the chain has decision $\in$ {accepted, conditional}.
S2	ChainMonotonicity	Safety	`consentChain` and `adherenceTrail` are append-only; lengths never decrease.
S3	AdherenceAnchored	Safety	Every AdherenceEvent references a valid ConsentRecord index.
S4	SkillRequiresPermit	Safety	A skill call is always preceded by at least one AdherenceEvent with decision = permit on an undisputed claim.
S5	ConditionalGating	Safety	A disputed claim always produces deny or escalate, never permit.
S6	NoDisputedPermit	Safety	No adherence event for a disputed claim carries a permit decision.

ID	Property	Kind	Statement
S7	NoSkillOnCapabilityDrift	Safety	No skill call occurs when the caller's capability fingerprint has changed since the ConsentRecord was created.
S8	GovernanceAlwaysReviews	Safety	Every re-consent record (index > 1 in the chain) has a governance tier set. No re-consent bypasses the governance agent.
S9	HumanRequiredHonoured	Safety	A re-consent with tier = `human_required` only completes after the human principal has acted.
L1	EventualReConsent	Liveness	Under weak fairness, any staleness (version bump or capability change) eventually leads to a new ConsentRecord.
L2	EventualCapReConsent	Liveness	A capability bump eventually leads to re-consent. The agent does not operate indefinitely under stale reasoning.

Properties S1–S4 establish the core consent-before-action guarantee. S5–S6 formalise conditional gating for reduced-permission operation. S7 addresses agent ephemerality: capability drift blocks all skill invocation until re-consent. Note that `caller_capability_hash` is self-reported by the caller; the callee cannot independently verify it. This is a known limitation — see §6.1. S8 and S9 formalise the tiered escalation model from §3.5: every re-consent passes through the governance agent (S8), and when the governance agent determines the change requires human review, the system blocks until the human acts (S9). Together, S8 and S9 ensure that the governance agent is a mandatory checkpoint — not an optional convenience, while still allowing immaterial changes to be auto-approved without alert fatigue. L1 and L2 ensure the protocol does not deadlock after either a policy version bump or a capability change.

The model was checked with TLC using `MaxVersions = 3`, `MaxAdherenceEvents = 4`, and `MaxCapVersions = 2`, covering 3 policy version bumps, 2 capability changes, and 4 adherence events per consent epoch. All eleven properties hold with zero violations across the reachable state space. TLC performs bounded model checking: it exhausts the state space within the declared constant bounds but does not constitute a proof for arbitrary parameter values. The bounds were chosen to cover realistic deployment scenarios (few policy versions, moderate event volume); unbounded verification would require inductive proof techniques

beyond the scope of this paper.

3.7 Audit Projection: From Machine Records to Plain English

The consent chain and adherence trail are machine-readable by design — structured for agents to produce, verify, and query. But the ultimate consumers of this data are human: compliance officers, legal teams, regulators, and the principals themselves. A machine-readable audit trail that requires an engineer to interpret defeats half its purpose.

The model defines an audit projection layer: a `GenerateAuditReport` RPC that walks the consent chain and adherence trail for a given caller-callee pair over a time range and produces a structured, plain-English report. The report has three components:

Executive summary. A 2–3 paragraph narrative covering the entire audit window. Example:

> *Between March 1 and March 5 2026, Agent X operated under two versions of Agent Y's usage policy (v2.0.0 and v2.1.0). Agent X accepted v2.0.0 unconditionally and conditionally accepted v2.1.0, disputing the new prohibition on session data aggregation (§3.4.2). Of 47 skill invocations, 45 were permitted and 2 were denied under §3.4.2. The transition from v2.0.0 to v2.1.0 was auto-approved by the governance agent (no new prohibitions beyond existing mandate). The human principal was not required to act.*

Chronological timeline. Each `ConsentRecord` and `AdherenceEvent` is rendered as a timestamped narrative entry. Consent entries describe what was agreed, which claims were disputed, what escalation tier was applied, and whether the human principal was involved. Adherence entries describe which clause was evaluated, what decision was reached, and the agent's reasoning, quoted verbatim from the `reasoning` field. Events are cross-referenced by `consent_record_id` and `adherence_event_id` so that any narrative claim can be traced back to the underlying machine record.

Per-claim and per-version summaries. Statistical breakdowns showing, for each claim: how many times it was evaluated, how many permits, denials, and escalations it produced, and a plain-English characterisation of its enforcement history. Per-version summaries show which policy version was active when, what the consent decision was, and how many adherence events occurred under it.

The projection is *derived*. It contains no data that is not already in the chain. It adds no new trust assumption. An auditor who doubts the narrative can verify every statement by following the cross-references to the underlying signed records. The plain-English layer exists for accessibility; the machine records remain the authoritative source.

This is the mechanism that closes the gap between protocol-level consent (useful

to engineers) and regulatory-grade auditing (useful to everyone else). A regulator asking "did this agent honour §3.4.2?" gets a plain-English answer with a citation trail, not a JSON dump. The chronological timeline and per-claim summaries directly satisfy the "meaningful information about the logic involved" requirement of GDPR Recital 71 and Arts. 13(2)(f)/14(2)(g), which oblige controllers to disclose the logic, significance, and envisaged consequences of automated decision-making under Art. 22. No existing agent protocol provides the clause-level reasoning these provisions anticipate.

3.8 Consent Categories: Asymmetric Sensitivity by Data Type and Usage

A blanket "I agree" treats all policy claims as equally important to the human principal. They are not. A user uploading a passport photo cares far more about how the photo is stored and processed than about the retention of their name. The same user at a pharmacy has no meaningful concern about prescription data being recorded (that is the *purpose* of the interaction) but would object to that data being shared with an advertising network. A parent with dependants may refuse *any* data sharing involving minors, regardless of context. Sensitivity is not uniform across data types; it is asymmetric, contextual, and personal.

But sensitivity is not just about *what kind of data*. It is equally about *what is being done with it*. A user may accept that a cosmetic surgeon stores their photos (the service cannot function otherwise) but refuse to allow those photos to be shared with a third party or used to train a model. Storage, access, sharing, and action are fundamentally different operations with different risk profiles. A consent model that only classifies data by type misses half the picture.

The model introduces two orthogonal classification axes on each `PolicyClaim`, and allows principals to express preferences across the resulting matrix.

Axis 1: Data categories. Each `PolicyClaim` carries a `category` field drawn from a controlled vocabulary of data types:

Category	Examples
Biometric	Facial images, fingerprints, voice prints, retinal scans
Health	Diagnoses, prescriptions, lab results, genetic data
Financial	Bank accounts, transactions, credit scores, payment instruments
Location	GPS coordinates, IP-derived location, travel history
Behavioural	Browsing history, app usage, purchase patterns
Identity	Name, email, phone, government IDs
Communications	Message content, call logs, email bodies

Category	Examples
Minor/Dependent	Any data concerning children or legal dependants
Operational	Non-personal metrics, model outputs with no PII

Axis 2: Usage dimensions. Each `PolicyClaim` also carries a `dimension` field classifying what operation is being performed:

Dimension	What it covers	Regulatory reference
Storage	Data persisted beyond the session: databases, caches, logs, backups	GDPR Art. 5(1)(e) — storage limitation
Access	Data read or displayed within the service boundary	—
Third-party sharing	Data transmitted outside the caller-callee pair: subprocessors, ad networks, downstream agents	GDPR Art. 28, 44–49; CCPA §1798.140(ah)
Automated decision	Data used as input to decisions affecting the principal: credit scoring, content filtering, risk assessment	GDPR Art. 22 — automated individual decision-making
Training	Data used to train, fine-tune, or evaluate ML models	Often the most contentious — principals frequently accept storage but refuse training
Aggregation	Data combined with other principals' data for analytics	Acceptable when anonymised, not when identifiable
Cross-context use	Data collected in one context used in another (health → insurance)	GDPR Art. 5(1)(b) — purpose limitation
Deletion/portability	Obligations to delete or export data on request	GDPR Art. 17, 20

Categories are intentionally coarse: they group claims by *kind*, not specific asset. Dimensions are intentionally broad, grouping operations by *type*, not specific API call. Fine-grained classification remains in the ODRL-aligned `action` and `asset` fields. The purpose is to give human principals a vocabulary for expressing what they care about without requiring them to understand ODRL semantics.

Category-dimension preferences. The human principal expresses a sensitivity level per (category, dimension) pair: **LOW** (auto-approve), **MEDIUM**

(notify but do not block), or **HIGH** (always escalate to me). These preferences are recorded on the `ConsentRecord` as `category_preferences` and form a matrix:

Example: cosmetic surgery app.

	Storage	Access	3P Sharing	Training	Auto. Decision
Biometric	HIGH	MEDIUM	HIGH	HIGH	HIGH
Health	MEDIUM	LOW	HIGH	HIGH	HIGH
Identity	LOW	LOW	LOW	LOW	LOW
Financial	LOW	LOW	MEDIUM	HIGH	MEDIUM

The same principal at a **pharmacy** might express:

	Storage	Access	3P Sharing	Training
Health	LOW	LOW	HIGH	HIGH
Identity	LOW	LOW	LOW	LOW
Financial	LOW	LOW	MEDIUM	HIGH

Note the difference: Health × Storage is HIGH at the surgeon (procedures are elective and stigma-sensitive) but LOW at the pharmacy (prescriptions are the point of the interaction). Health × Third-Party Sharing is HIGH in both contexts. The principal never wants their health data shared regardless of where it was collected.

The preferences travel with the `ConsentRecord`, not the principal's global profile, because sensitivity is context-dependent. A principal may set the matrix differently for every callee agent.

Default and override semantics. A preference entry with an unspecified dimension acts as the default for *all* dimensions of that category. A specific (category, dimension) entry overrides it. For example: `(BIOMETRIC, unspecified) = HIGH` sets all biometric operations to high sensitivity, but `(BIOMETRIC, ACCESS) = LOW` carves out an exception for the surgeon viewing the photo during consultation.

Integration with tiered escalation. Category-dimension preferences feed directly into the governance agent's materiality assessment (§3.5). When evaluating a re-consent event, the governance agent checks each changed claim's `(category, dimension)` pair against the principal's preference matrix:

- A new prohibition on biometric third-party sharing where `(BIOMETRIC, THIRD_PARTY_SHARING) = HIGH` → `HUMAN_REQUIRED`, regardless of other materiality signals.
- A tightened constraint on operational storage where `(OPERATIONAL, STORAGE) = LOW` → eligible for `AUTO_APPROVED`.

- A relaxed permission on financial training where `(FINANCIAL, TRAINING) = HIGH` → `HUMAN_REQUIRED`. The principal does not want their payment data used to train models.
- An added obligation for health data deletion where `(HEALTH, DELETION_PORTABILITY) = MEDIUM` → `GOVERNANCE_REVIEWED` with principal notification.

This means the governance agent's materiality table (§3.5) gains an additional row:

Factor	Material if...
Category-dimension sensitivity	Claim's `(category, dimension)` pair has `HIGH` sensitivity in principal's preferences

The preference is a *floor*, not a ceiling. A claim in a LOW cell can still be escalated if other materiality factors trigger it (e.g., a `rule_type` change from permission to prohibition). But a claim in a HIGH cell is *always* escalated because the principal has explicitly said "I care about this specific operation on this specific kind of data."

Minor/dependent data. The `MINOR_DEPENDENT` category deserves special note. A principal who indicates they have dependants (by setting this category to HIGH across all dimensions) is expressing a blanket constraint: no claim affecting minor data may be auto-approved, ever — not storage, not access, not sharing, not training. This aligns with COPPA, GDPR Article 8, and similar statutes that impose heightened protections for children's data. The agent cannot reason its way around this. The principal's preference is a hard gate across every dimension.

---

4. Protocol Instantiations

4.1 Agent2Agent (A2A) Protocol

A2A is an open protocol for agent-to-agent communication, now governed by the Linux Foundation with Google, Microsoft, AWS, Salesforce, SAP, and ServiceNow as founding members [11]. Its AgentCard (a JSON document published at `/.well-known/agent.json`) advertises an agent's capabilities, skills, and authentication requirements.

A2A already supports protocol extensions via the `capabilities.extensions` array [4]. Each extension is declared with a `uri`, `description`, and `required` flag. This is the mechanism used by the Agent Payments Protocol (AP2) to advertise payment capability [12]. We use the same mechanism.

Step 1: Declare the extension in the AgentCard

```
{
  "capabilities": {
    "extensions": [{
      "uri":
      "https://github.com/ravikiran438/agent-consent-protocol/v1"
      ,
      "description": "Supports versioned usage policy and consent
      auditing.",
      "required": true
    }]
  },
  "usage_policy": {
    "version": "2.1.0",
    "document_uri":
    "https://callee.example.com/.well-known/usage-policy.json",
    "document_hash": "sha256:a3f5c2...",
    "effective_date": "2026-02-27T00:00:00Z",
    "acceptance_required": true,
    "acceptance_endpoint":
    "https://callee.example.com/acap/consent",
    "natural_language_uri": "https://callee.example.com/terms"
  }
}
```

Step 1b: Annotate skills with governing claims

Each skill in the AgentCard carries a `policy_claims` array listing the claim IDs that govern its use, mirroring the MCP `policyClaims` annotation (§4.2). This allows the calling agent to evaluate only the relevant claims before invoking each skill:

```
{
  "skills": [{
    "name": "analyse_dataset",
    "policy_claims": ["claim-data-retention",
    "claim-aggregation-prohibition"]
  }]
}
```

Without this mapping, the callee has no mechanism to enforce claim-level gating for conditional consent (§3.2); it would not know which skills to block when specific claims are disputed.

Step 2: Consent handshake before first skill call

The calling agent fetches the `PolicyDocument`, verifies its hash, parses every `PolicyClaim`, and POSTs a `ConsentRecord` to the `acceptance_endpoint` before invoking any skill. The callee verifies that the record covers every claim and that the `policy_hash` matches the current document. Only then does it permit

skill invocation.

Step 3: Per-action adherence recording

Before each skill call, the calling agent evaluates the relevant claims and POSTs an `AdherenceEvent` to the callee. The callee appends the event to the adherence trail for that consent record. The complete flow is shown in Figure 3.

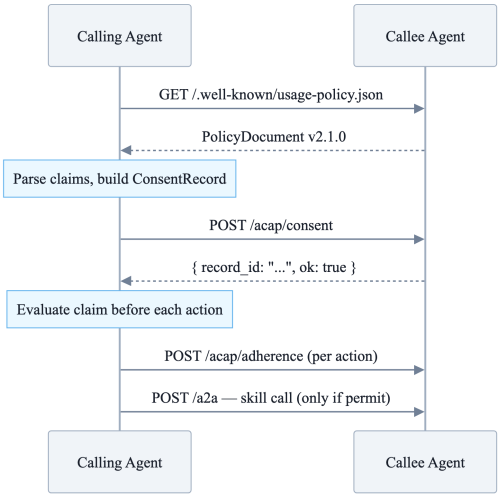

Figure 3: ACAP consent and adherence sequence.

Enforcement modes. The `POST /acap/adherence` endpoint operates in one of two modes, declared in the AgentCard's `usage_policy` object:

Mode	Caller decides	Callee role	Use case
`local`	Yes — caller evaluates claims and acts	Audit record only	Low-stakes, high-throughput
`delegated`	Callee returns permit/deny	Policy enforcement point	High-stakes, regulated

In `local` mode, the adherence event is fire-and-forget: the caller records its own decision and the callee appends it to the audit trail. In `delegated` mode, the callee evaluates the event and returns an enforcement decision; the caller MUST NOT invoke the skill until it receives a `permit` response. Delegated mode adds a round trip but gives the callee veto authority, appropriate for regulated interactions where the callee bears compliance obligations.

Version bump handling. When the callee publishes a new `PolicyDocument`, the calling agent detects the version change on the next AgentCard fetch

(by comparing `usage_policy.version` against its cached consent record's `policy_version`). It fetches the new document, identifies changed claims via `since_version`, and creates a new `ConsentRecord` with `prev_id` pointing to the now-invalidated record. The old record is never deleted.

4.2 Model Context Protocol (MCP)

MCP governs communication between an orchestrator agent (client) and tools or data sources (servers) [5]. The November 2025 MCP specification explicitly identifies "version tracking and policy enforcement" as first-class future concerns [6], acknowledgement that the gap exists.

The model maps to MCP as follows. The MCP `initialize` handshake response includes a `capabilities` object. We add a `usagePolicy` capability:

```
{
  "capabilities": {
    "tools": {},
    "usagePolicy": {
      "version": "1.0.0",
      "documentUri":
      "https://server.example.com/.well-known/usage-policy.json",
      "documentHash": "sha256:b7f3a1...",
      "acceptanceRequired": true
    }
  }
}
```

Individual tool definitions can carry a `policyClaims` annotation listing the claim IDs that govern their use:

```
{
  "name": "aggregate_user_data",
  "description": "Aggregates user data across sessions.",
  "policyClaims": ["claim-pii-aggregate-prohibition"],
  "inputSchema": { ... }
}
```

This allows calling agents to evaluate only the relevant subset of claims before invoking each tool, rather than re-evaluating the entire policy document per call. The consent and adherence records use the same schema as the A2A instantiation.

The MCP instantiation is lighter because MCP's trust boundary is typically within a single organisation (orchestrator to owned tools). Cross-organisational A2A calls carry higher legal weight and warrant the fuller handshake.

---

5. Related Work

Several recent proposals address agent governance from different angles. We survey them here, grouping by the layer each addresses.

Single-agent governance. OAGS [13] defines five governance primitives (deterministic identity, declarative policy, runtime enforcement, structured audit evidence, cryptographic verification) but is explicitly "local-first." It governs what one agent may do in its own environment, not what two agents agree to when they interact. OpenMandate [16] takes a similar single-agent stance, separating policy from execution using declarative YAML mandates. Both address the right abstraction level but stop at the organisational boundary. Policy Cards [17] make an individual agent's constraints inspectable through machine-readable governance artefacts. The `PolicyDocument` is the callee's equivalent of a Policy Card, but `ConsentRecord` and `AdherenceEvent` capture the bilateral negotiation and ongoing compliance that Policy Cards don't cover.

Runtime enforcement. PCAS [18] compiles Datalog-derived policies into instrumented agents that are policy-compliant by construction. A deployment could use PCAS for local enforcement and ACAP for cross-agent consent; they address different layers. MI9 [20] focuses on drift detection, identifying when agents deviate from expected behaviour. MI9 detects non-compliance after the fact; ACAP provides the evidentiary basis (what was agreed, what was checked) that drift detection consumes.

Governance architecture. GaaS [19] proposes an external governance agent supervising other agents at runtime. The tiered escalation model in §3.5 is an instance of this pattern, specialised to consent-change materiality assessment. The AIGA Internet-Draft [15] proposes tiered risk-based governance covering action authorisation and audit logging. It governs *what* an agent does; ACAP governs *under what agreed terms*.

Financial services. FINOS AI Governance Framework v2.0 [14] defines MI-21 (Agent Decision Audit and Explainability) with tiered audit logging up to cryptographic tamper-evidence. MI-21 is conceptually close to the adherence trail but explicitly silent on consent versioning [14]. A FINOS contribution is in preparation.

Privacy and consent standards. IEEE P7012 [21] defines machine-readable personal privacy terms, closely aligned with the category-dimension preference matrix in §3.8. P7012 addresses the human-to-service relationship; ACAP addresses the agent-to-agent relationship where human preferences must propagate through delegation chains. W3C DPV v2.2 [22] provides standardised vocabularies for data processing activities. Our data categories (§3.8) are compatible with DPV's taxonomies but serve a different role: they form the axes of a preference matrix governing consent decisions *before* processing, not describing processing *after*. The Kantara Consent Receipt [23] established early principles (consent as receipt, machine-readability, user portability) that inform the `ConsentRecord`

design. We extend the receipt model to agent-to-agent context with per-action adherence and versioned chains.

OAuth ecosystem. UMA 2.0 [27] enables resource owners to control delegated access through an authorisation server. UMA governs *who may access which resource*; ACAP governs *what a calling agent agreed the callee may do* and whether it honoured those terms at every action. A UMA server could require a valid ACAP `ConsentRecord` as a precondition for token issuance. RFC 9396 (Rich Authorization Requests) [26] extends OAuth with structured authorization data beyond binary scopes. RAR lets a client express *what* it wants to do, but it is a request-time mechanism with no versioning, no per-action adherence, and no re-consent on policy change.

Audit trails. AuditableLLM [24] contributes hash-chain audit trails for language model interactions. Our chain structure shares the pattern but applies it to consent and policy adherence across agent boundaries.

Foundational analysis. Rida [25] examines what consent means when an AI agent, not a human, is the party clicking "I agree." That analysis directly motivates the present work.

ODRL 2.2 [9] provides the vocabulary we use for `PolicyClaim.action` and `PolicyClaim.asset`. We treat ODRL as a vocabulary convention, not a normative dependency. ODRL is a rights *expression* language: it describes what is permitted, prohibited, or obligated. It is not a consent *lifecycle* protocol. It has no `ConsentRecord`, no `AdherenceEvent`, no versioned chain, and no capability-bound invalidation. ACAP uses ODRL vocabulary for claim semantics and addresses the consent lifecycle layer that ODRL does not cover.

Summary of differentiation. Six contributions distinguish this work: (1) the proof-of-adherence framing, shifting from one-time acceptance to per-action, per-clause adherence events with reasoning trails; (2) the inter-agent consent chain, a versioned linked-list audit structure *between* two agents; (3) capability-bound consent, tying consent validity to the agent's own configuration; (4) the (category $\times$ dimension) sensitivity matrix for asymmetric data-type and usage-operation preferences; (5) regulatory context propagation, with structured floors from HIPAA, GDPR, PCI-DSS that flow through delegation; (6) `PolicyDiff` for version-bump re-consent, making human escalation actionable rather than ceremonial.

---

6. Discussion

6.1 What This Does Not Solve

The `reasoning` field on `AdherenceEvent` is free-form natural language. This is a deliberate choice for v0.1: it makes the field useful to human auditors immediately. It also means reasoning strings are not machine-comparable: two

agents may evaluate the same clause differently and produce non-equivalent reasoning strings with no way to detect the discrepancy. A structured reasoning format (constraint logic, ODRL expressions) would enable automated auditing but would substantially raise the implementation barrier. We defer this.

The model assumes calling agents act in good faith when populating `parsed_claims` and `reasoning`. A malicious agent could record `understood: true` for a prohibition and then violate it. This is worse than the current state (it produces false evidence), but it also creates a clearer liability standard. An agent that explicitly acknowledged a prohibition and violated it has acted in demonstrable bad faith, which is legally more tractable than silent non-compliance under a clicked-through ToS.

Threat model. The self-attestation weakness is real and should be stated plainly: ACAP does not prevent non-compliance. It makes non-compliance *detectable after the fact* and *legally attributable*. Three mitigations are planned or possible:

1. **Callee-side enforcement.** The callee can independently verify compliance for observable actions (e.g., checking whether data was stored beyond a session by monitoring its own APIs). ACAP provides the evidentiary framework; enforcement is orthogonal.
2. **Third-party audit.** The chain anchoring mechanism (§3.4) allows an independent auditor to verify that the consent chain was not tampered with. Combined with callee-side logs, this creates a two-party evidence model where neither party alone can falsify the record.
3. **Structured reasoning (v0.2).** A future version may replace the free-form `reasoning` field with a constrained reasoning format (e.g., ODRL constraint expressions) that enables automated compliance checking, moving from "trust but verify" to "verify then trust."

S7 and the self-reported capability hash. Property S7 (NoSkillOnCapabilityDrift) guarantees that a capability change triggers re-consent. However, the `caller_capability_hash` is self-reported. A caller that silently upgrades its model without updating the hash would bypass S7 entirely. The callee has no mechanism to independently verify the caller's capability fingerprint. This is structurally analogous to the self-attestation weakness above: ACAP makes the *claim* auditable but cannot force the claim to be *truthful*. Remote attestation or trusted execution environments could close this gap in future work.

Manufactured deniability. The inverse risk of self-attestation is that a caller could populate `reasoning` fields *designed to establish plausible deniability* — recording a permit decision with reasoning that would appear reasonable to a human auditor but that the agent never actually evaluated. This is worse than no audit trail because it creates affirmative evidence of compliance where none exists. The structured reasoning format planned for v0.2 (item 3 above) partially addresses this by constraining what counts as valid reasoning, but the fundamental tension between self-reporting and verification remains.

The honest comparison is not "ACAP vs. perfect enforcement" but "ACAP vs. the current state," which is no consent record at all.

6.2 The Three-Hop Problem

The model handles the caller-callee pair. It does not yet address delegation chains: Agent A calls Agent B which calls Agent C. In this case, Agent B is simultaneously a callee (to A) and a caller (to C). The consent chain for A→B and the consent chain for B→C are independent. What is not specified is how the consent records for these two chains are linked when a human principal needs to reconstruct the full delegation path.

A partial solution: the `ConsentRecord` could carry a `delegation_chain` field: an ordered list of `(agent_id, consent_record_id)` pairs representing the upstream delegation path that led to this interaction. This makes the delegation path *traversable* without solving the harder problem of policy conflict resolution across chains. We defer conflict resolution but note that traversability alone is valuable: a regulator asking "how did my data get to Agent C?" can walk the chain A→B→C even if the consent semantics at each hop are independent.

6.3 Agent Ephemerality and Capability-Bound Consent

Human users persist. They create accounts, accumulate consent history, and the consent chain tracks a stable identity across years. Agents are different in kind: they are spawned, updated, and terminated. An agent instance may exist for minutes or hours, and the next instance, even if it shares the same `caller_agent_id`, may run a different model, carry different tools, or operate under a different context window.

This creates a problem that human consent systems never face: **the reasoning entity that consented may not be the reasoning entity that acts**. An agent running GPT-4o that recorded `understood: true` for a prohibition on data aggregation may be replaced by an instance running a fine-tuned variant that interprets "aggregation" differently. The `ParsedClaim` entries in the original `ConsentRecord` are no longer trustworthy. Not because they were falsified, but because the entity that produced them no longer exists.

The model addresses this with `caller_capability_hash`: a SHA-256 fingerprint of the agent's model identifier, tool manifest, and reasoning configuration (i.e., system prompt, temperature, and any chain-of-thought or retrieval-augmented generation settings that influence how the agent interprets policy claims), recorded on every `ConsentRecord`. Three re-consent triggers are defined:

Trigger	Cause	Effect
`POLICY_BUMP`	Callee publishes new PolicyDocument	Re-fetch, diff, re-consent
`CAPABILITY_CHANGE`	Caller's capability hash changes	Re-evaluate all ParsedClaims

Trigger	Cause	Effect
`PRINCIPAL_CHANGE`	Human principal identity changes	Full re-consent with new principal_id

The `valid_until` field supports sentinel values that control invalidation granularity: `"on_version_bump"` (callee-side changes only), `"on_capability_change"` (caller-side changes only), or `"on_any_change"` (either side). This lets callees choose their risk tolerance. A financial services agent may require `"on_any_change"`, while a low-stakes utility agent may accept `"on_version_bump"`.

Persistence model. The consent chain is stored by the callee. It is their audit trail and their legal protection. The chain is keyed by `caller_agent_id` (a DID or HTTPS URL bound to the principal or organisation, not the process), so new agent instances inherit the chain if they share the identity. Both parties SHOULD replicate the chain, and a neutral audit endpoint is RECOMMENDED for high-stakes interactions. The chain itself is append-only; agent death does not delete records.

6.4 Agents Are Not the Regulated Entity

HIPAA does not govern agents. PCI-DSS does not govern agents. GDPR does not govern agents. These frameworks govern *organisations* (covered entities, merchants, data controllers) and the humans who operate them. An agent calling a healthcare API is not a HIPAA-covered entity; the hospital that deployed the agent is. The agent is an instrument of the regulated party, not a regulated party itself.

But the agent must *behave as if* it were subject to the framework, because the compliance obligation flows through delegation. When a hospital deploys an agent to interact with an analytics service, the hospital's HIPAA obligations constrain how that agent handles Protected Health Information, regardless of what the analytics service's policy permits and regardless of what the human principal personally prefers. The regulatory framework is a *floor* that neither the principal's category preferences nor the callee's policy claims can lower.

The model captures this via `RegulatoryContext`: a structured declaration on both the `PolicyDocument` (callee's compliance context) and the `ConsentRecord` (principal's compliance context). Each context declares:

- **Framework:** which regulatory regime applies (GDPR, HIPAA, PCI-DSS, EU AI Act, CCPA, COPPA, etc.).
- **Role:** the declaring party's role under the framework (data controller, covered entity, merchant, deployer, etc.).
- **Obligations:** specific behavioural constraints the framework imposes, expressed as (category, dimension) pairs with a minimum sensitivity level.

The governance agent computes the *effective sensitivity* for each (category, dimension) pair as the maximum of three inputs, using the total order LOW < MEDIUM < HIGH:

```
effective_sensitivity(category, dimension) =
    max(
        principal_preference(category, dimension),    -- §3.8
        callee_obligation(category, dimension),       --
        PolicyDocument
        caller_obligation(category, dimension)        --
        ConsentRecord
    )
```

When a source does not declare a value for a given (category, dimension) pair, it defaults to LOW (i.e., no constraint). This ensures the max() is always defined over three inputs.

Example: hospital calling an analytics agent.

Source	(HEALTH, THIRD_PARTY_SHARING)	Result
Principal preference	LOW ("I don't care")	—
Callee policy	PERMISSION (sharing allowed)	—
HIPAA §164.502(b) (caller obligation)	HIGH (minimum necessary rule)	**Effective: HIGH**

The principal may not care. The callee may permit it. But HIPAA says no. The regulatory floor wins, and the governance agent escalates to `HUMAN_REQUIRED`. The agent cannot reason its way around a statutory obligation. It can only flag it for the human who *is* the regulated party.

Union semantics. When both parties declare regulatory contexts, the governance agent applies the *union* of obligations. If the callee operates under GDPR (as a data processor) and the caller operates under HIPAA (as a covered entity), both sets of obligations apply. For any given (category, dimension) pair, the stricter constraint wins. This means cross-border, cross-framework interactions automatically inherit the most protective standard, which is the legally correct behaviour.

Why not just use ODRL constraints? The `constraint` field on `PolicyClaim` can express some regulatory requirements (e.g., `odrl:purpose isNot behavioral_profiling`). But regulatory obligations are properties of the *parties*, not the *policy*. A callee's policy may permit data sharing; the caller's HIPAA obligation prohibits it. The constraint lives on the wrong side of the protocol. Regulatory context on the `ConsentRecord` puts it where it belongs: with the party that bears the compliance obligation.

This design deliberately avoids making the agent a compliance engine. The agent does not interpret HIPAA or GDPR. It propagates structured obligations that humans (or governance agents operating under human- defined delegation policies) declared. The legal interpretation happens at configuration time, not at runtime. The agent enforces floors; it does not adjudicate law.

6.5 The EU AI Act Deadline

The EU AI Act (Regulation (EU) 2024/1689) enforces in phases. Prohibited AI practices took effect in February 2025. General-purpose AI model obligations (Title V) applied from August 2025. The obligations most relevant to this paper — Art. 14 (human oversight for high-risk AI systems) and Art. 50 (transparency obligations including disclosure of AI-generated content and automated decision-making) — take effect on August 2, 2026, for high-risk systems listed in Annex I and Annex III [2]. A fourth phase covering high-risk systems under Annex I, Appendix II follows in August 2027.

Art. 14 requires that high-risk AI systems "be designed and developed in such a way . . . that they can be effectively overseen by natural persons." This obligation applies specifically to AI systems classified as high-risk under Annex I and Annex III (e.g., credit scoring, employment screening, biometric identification) — not to every agent deployment. For autonomous agents operating in these high-risk contexts, effective oversight requires, at minimum, that the principal can determine what policies the agent agreed to, what clauses it evaluated at each action, and what reasoning it applied. The consent chain and adherence trail provide exactly this audit surface. ACAP is valuable beyond high-risk contexts, but Art. 14 provides the strongest regulatory mandate for its adoption.

Art. 50 requires deployers of AI systems that interact with natural persons to ensure those persons are informed they are interacting with an AI system. The obligation applies at the human-facing boundary of the chain — where the AI system directly interacts with a natural person — not necessarily at every intermediate agent-to-agent hop. In multi-agent chains, where Agent A calls Agent B which calls Agent C on behalf of a human, the `delegation_chain` field proposed in §6.2 and the audit projection layer in §3.7 together make that human-facing boundary traceable, so the deployer at the boundary can satisfy Art. 50 with full knowledge of the upstream delegation path.

As of this writing, the EU AI Office has published no technical guidance addressing consent mechanisms for autonomous agent-to-agent transactions [3]. The ACAP model is designed to satisfy the spirit of Arts. 14 and 50 in advance of such guidance, so that implementations are not built retroactively under deadline pressure.

6.6 Open Questions

We identify six questions the community should resolve:

1. **Self-attestation vs. verification.** `ParsedClaim.understood` is currently self-attested. Should there be a verifiable proof that the calling agent processed the claim text? Zero-knowledge proofs of document processing are theoretically possible but practically expensive.

2. **Grace periods.** `valid_until: "on_version_bump"` blocks all skill invocation immediately on version change. For long-running tasks, an immediate block may be disruptive. A grace period (e.g., complete in-flight tasks, block new initiations) needs specifying.

3. **Consent chain for competing policies.** When Agent A is governed by its own principal's policy *and* must accept callee Agent B's policy, these policies may conflict. The model records both but provides no conflict resolution mechanism. This is particularly acute in regulated sectors where both parties operate under regulatory mandates.

4. **Capability fingerprint granularity.** The `caller_capability_hash` is defined as a SHA-256 digest of model identifier, tool manifest, and reasoning configuration. In practice, what constitutes a "material" capability change is ambiguous. A minor prompt template update may not affect policy reasoning, while a model version bump almost certainly does. The current design treats any fingerprint change as a re-consent trigger; a future version may introduce a capability diff analogous to `PolicyDiff` to distinguish material from immaterial changes.

5. **Bootstrapping and migration.** The protocol assumes the calling agent knows to fetch the usage policy before its first call. Two adoption gaps remain: (a) when a callee does not yet advertise `acceptance_required: true`, the calling agent has no trigger to initiate the handshake; (b) when ACAP is adopted by an agent pair that has already been interacting, there is no retroactive consent record. A migration path would define an implicit permissive policy (version `0.0.0`, zero claims) that creates a baseline consent record, with `valid_until: "on_version_bump"` to force re-consent when the callee publishes its first real policy.

6. **Regulatory obligation authoring.** The `ComplianceObligation` messages in `RegulatoryContext` are currently authored by the deploying organisation at configuration time. This requires legal expertise to translate framework requirements into (category, dimension, minimum_sensitivity) tuples. A standardised library of pre-authored obligation sets per framework (e.g., "HIPAA covered entity, standard obligations") would reduce the barrier to adoption, but raises questions about who maintains and certifies such libraries. This is a governance problem, not a protocol problem.

---

7. Conclusion

The consent model we inherited from human auth systems (a timestamped acceptance record) is inadequate for autonomous agent protocols. It proves acceptance. It cannot prove adherence. The gap between those two words is the gap between a click and an obligation.

Agents are capable of something humans are not: parsing policy documents, evaluating clauses at runtime, and producing reasoning trails for every action. The three primitives proposed here (`PolicyDocument`, `ConsentRecord`, `AdherenceEvent`) provide the infrastructure to make that capability useful. The versioned linked-list audit chain gives legal accountability to the calling agent's principal. The instantiations on A2A and MCP show the model works against current protocol standards without changes to their core architecture.

The timing matters. The EU AI Act's high-risk enforcement date (August 2, 2026) is five months away. The A2A specification is under active revision by a newly formed Linux Foundation working group. The FINOS AI Governance Framework has an open gap in MI-21. This is an unusually short window in which a protocol proposal can land in multiple active governance processes simultaneously.

The author built a version of this system for humans, years ago at a consumer technology company. Linked consent chains, immutable versions, the whole audit trail. The data showed users spent four seconds on the acceptance screen. The chain proved acceptance with precision; it proved nothing else. That experience is why this paper exists. Agents do not need four seconds. They need a protocol that asks them to actually read the document, evaluate each clause, and leave a record of what they decided and why. That is what ACAP provides.

This protocol governs entry. A companion protocol, Phala [28], is designed to address exit: whether the interaction served the principal, and whether that signal should modulate the learning behaviour of the agent network. Anumati governs the conditions under which an agent may act; Phala is intended to measure whether the action bore fruit. Together they are designed to bracket the full accountability lifecycle of every agent interaction.

References

[1] AI Agent Index 2025. arXiv:2602.17753, February 2026. (The report covers the 2025 landscape; the preprint was published in February 2026.)

[2] EU AI Act. Regulation (EU) 2024/1689. https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai

[3] EU AI Office. AI Act Service Desk — Frequently Asked Questions. https://ai-act-service-desk.ec.europa.eu/en/faq (As of March 2026, no FAQ or

guidance document addresses consent mechanisms for autonomous agent-to-agent transactions.)

[4] A2A Protocol Specification. https://a2a-protocol.org/latest/specification/

[5] Model Context Protocol Specification. https://modelcontextprotocol.io/specification/

[6] MCP November 2025 Changelog. https://modelcontextprotocol.io/specification/2025-11-25/changelog

[7] Uniform Electronic Transactions Act (UETA), §14. https://www.uniformlaws.org/

[8] Proskauer: Contract Law in the Age of Agentic AI, 2025. https://www.proskauer.com/blog/contract-law-in-the-age-of-agentic-ai-whos-really-clicking-accept

[9] W3C ODRL Information Model 2.2. https://www.w3.org/TR/odrl-model/

[10] RFC 7515: JSON Web Signature (JWS). https://datatracker.ietf.org/doc/html/rfc7515

[11] Linux Foundation: A2A Protocol Launch, February 2026. https://www.linuxfoundation.org/press/linux-foundation-launches-the-agent2agent-protocol-project

[12] AP2: Agent Payments Protocol. https://github.com/google-agentic-commerce/ap2

[13] Ngozo, J.F. Open Agent Governance Specification (OAGS). Sekuire, 2026. https://sekuire.ai/blog/introducing-open-agent-governance-specification

[14] FINOS AI Governance Framework v2.0. https://air-governance-framework.finos.org/

[15] Aylward, J. et al. AIGA: AI Governance and Accountability Protocol. IETF Internet-Draft, draft-aylward-aiga-1. https://datatracker.ietf.org/doc/draft-aylward-aiga-1/

[16] McDonough, R. OpenMandate: Governing AI Agents by Authority, Not Instruction. Law://WhatsNext, 2026. https://lawwhatsnext.substack.com/p/openmandate-governing-ai-agents-by

[17] Mavračić, J. Policy Cards: Machine-Readable Runtime Governance for Autonomous AI Agents. arXiv:2510.24383, 2025.

[18] Palumbo, N. et al. PCAS: Policy Compiler for Secure Agentic Systems. arXiv:2602.16708, 2026.

[19] Gaurav, S. et al. Governance-as-a-Service: A Multi-Agent Framework for AI System Compliance and Policy Enforcement. arXiv:2508.18765, 2025.

[20] Wang, C.L. et al. MI9: An Integrated Runtime Governance Framework for Agentic AI. arXiv:2508.03858, 2025.

[21] IEEE P7012: Standard for Machine-Readable Personal Privacy Terms. https://standards.ieee.org/ieee/7012/

[22] W3C Data Privacy Vocabulary (DPV) v2.2. https://w3c.github.io/dpv/dpv/

[23] Kantara Initiative: Consent Receipt Specification v1.1. https://kantarainitiative.org/

[24] Li, D., Yu, G., Wang, X. and Liang, B. AuditableLLM: A Hash-Chain-Backed, Compliance-Aware Auditable Framework for Large Language Models. Electronics, 15(1), 56. MDPI, 2025.

[25] Rida, C. When an AI Agent Says 'I Agree,' Who's Consenting? TechPolicy.Press, December 2025. https://www.techpolicy.press/when-an-ai-agent-says-i-agree-whos-consenting/

[26] RFC 9396: OAuth 2.0 Rich Authorization Requests. https://datatracker.ietf.org/doc/html/rfc9396

[27] Kantara Initiative. *User-Managed Access (UMA) 2.0 Grant for OAuth 2.0 Authorization*. January 2017. https://kantarainitiative.org/uma-specifications/

[28] Kadaboina, R. K. *Phala: Principal-Declared Welfare Feedback as a Protocol Extension for Autonomous Agent Networks*. In preparation, 2026.